WhatsApp-Integrated Smart Taxi Booking System

SOA Project Review-3

Under the Guidance of

K. Venkateswari

Professor,

Department of Computer Science and Engineering,

KLH CSE Bowrampet Campus

Team Members

Name: G. Tanay Tej

Roll Number: 2420030560

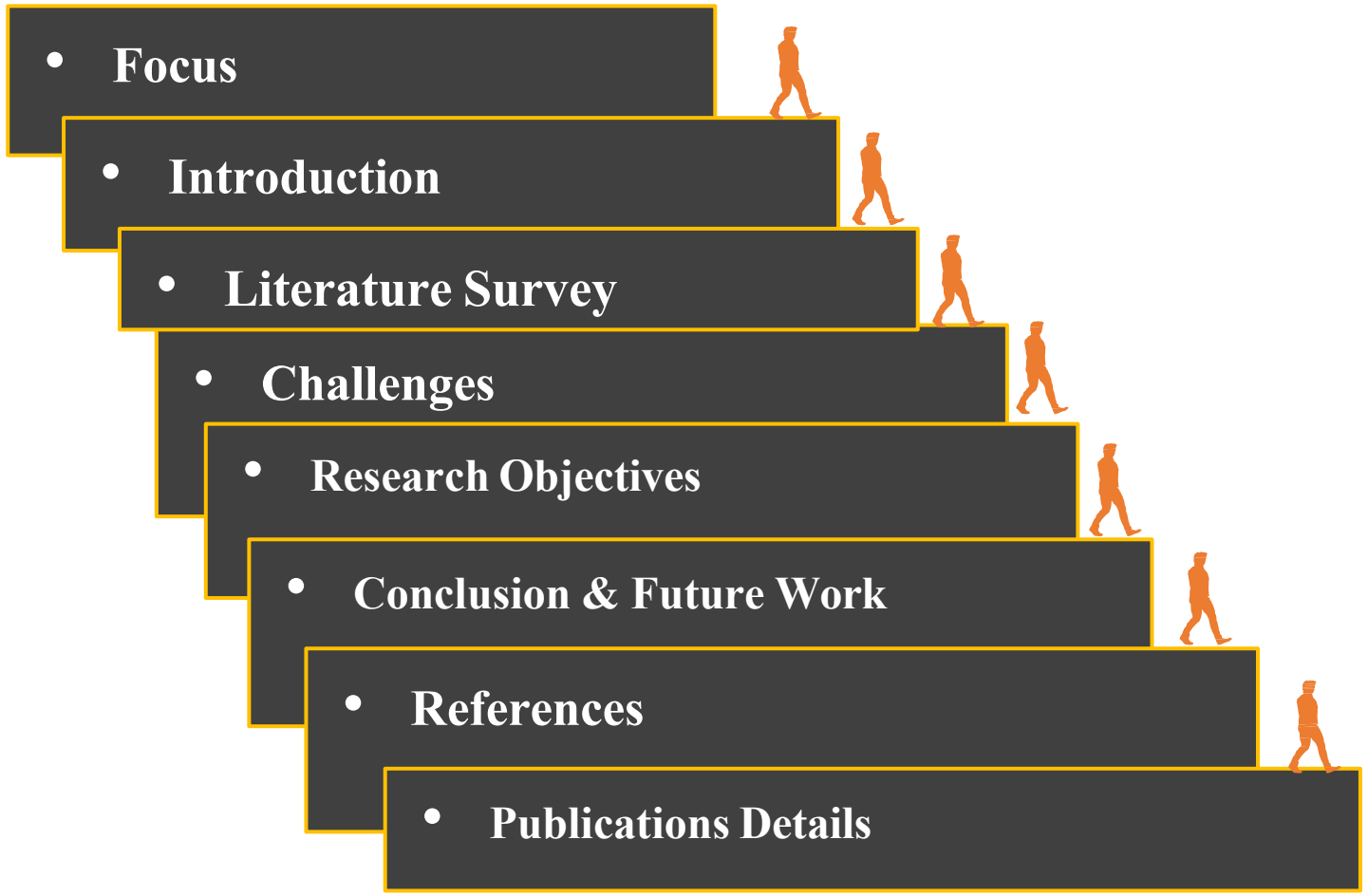
Name: G. Manikanta

Roll Number: 2420030366

Name: I. Vishnu Vardhan

Roll Number: 2420030513

Outline of Project Work



Problem Statement

Problem Statement:-

Traditional taxi booking systems often require users to install dedicated applications, create accounts, and navigate multiple screens to book a ride. This can be inconvenient for users who prefer simple and familiar communication platforms. Additionally, the lack of seamless real-time communication between passengers, drivers, and administrators can lead to delays and inefficient ride management.

The **WhatsApp-Integrated Smart Taxi Booking and Real-Time Ride Management System** aims to solve these challenges by providing a convenient taxi booking service through WhatsApp. The system enables users to book rides, share pickup and destination details, receive driver information, track ride status, and get booking updates through WhatsApp. It also helps drivers manage ride requests and allows administrators to efficiently monitor users, drivers, and bookings. The proposed system improves accessibility, communication, and overall efficiency in taxi booking and ride management.

Objectives

- **To provide easy taxi booking through WhatsApp** without requiring a separate mobile application.
- **To automate driver assignment** based on driver availability and location.
- **To enable real-time ride tracking** and provide live ride status updates.
- **To improve communication** between passengers, drivers, and administrators through WhatsApp notifications.
- **To manage taxi bookings efficiently**, including booking, acceptance, cancellation, and ride completion.
- **To provide fare estimation and trip details** for better transparency.
- **To develop an admin dashboard** for managing users, drivers, vehicles, and ride records.
- **To improve overall convenience, efficiency, and accessibility** of taxi transportation services.

Literature Survey

Authors	Title of the Paper	Methodology	Inferences	Limitation/Future work
Correia et al. [1]	A Generic Data-Driven Recommendation System for Large-Scale Regular and Ride-Hailing Taxi Services, Electronics, 2020	• Real-time taxi and customer data collection • Demand prediction • Taxi-to-region matching • Route planning	• Improves taxi allocation using real-time information. • Helps match available taxis with customer demand. • Supports efficient ride-hailing operations.	• Focuses mainly on taxi recommendation and allocation. • Does not provide WhatsApp-based booking. • Future work can integrate conversational booking channels.
Agarwal et al. [2]	The Impact of Ride-Hail Surge Factors on Taxi Bookings, Transportation Research Part C, 2022	• Machine-learning-based demand prediction • Analysis of ride-hailing and taxi booking data • Driver guidance and demand estimation	• Improved demand prediction can reduce vacant taxi time. • Better information can improve taxi utilization and trip efficiency.	• Focuses on pricing and demand prediction. • Does not address direct user communication through messaging platforms. • Future work can explore integrated booking and communication systems.
Chen et al. [3]	Real-Time Spatial-Intertemporal Pricing and Relocation in a Ride-Hailing Network, Operations Research, 2024	• Real-time demand and supply analysis • Dynamic pricing • Optimization using taxi network data	• Real-time decision-making improves the balance between drivers and passengers. • Dynamic strategies can improve ride-hailing efficiency under changing demand.	• Primarily focuses on pricing and vehicle relocation. • Does not include WhatsApp-based interaction or complete ride management. • Future work can combine real-time management with conversational booking.

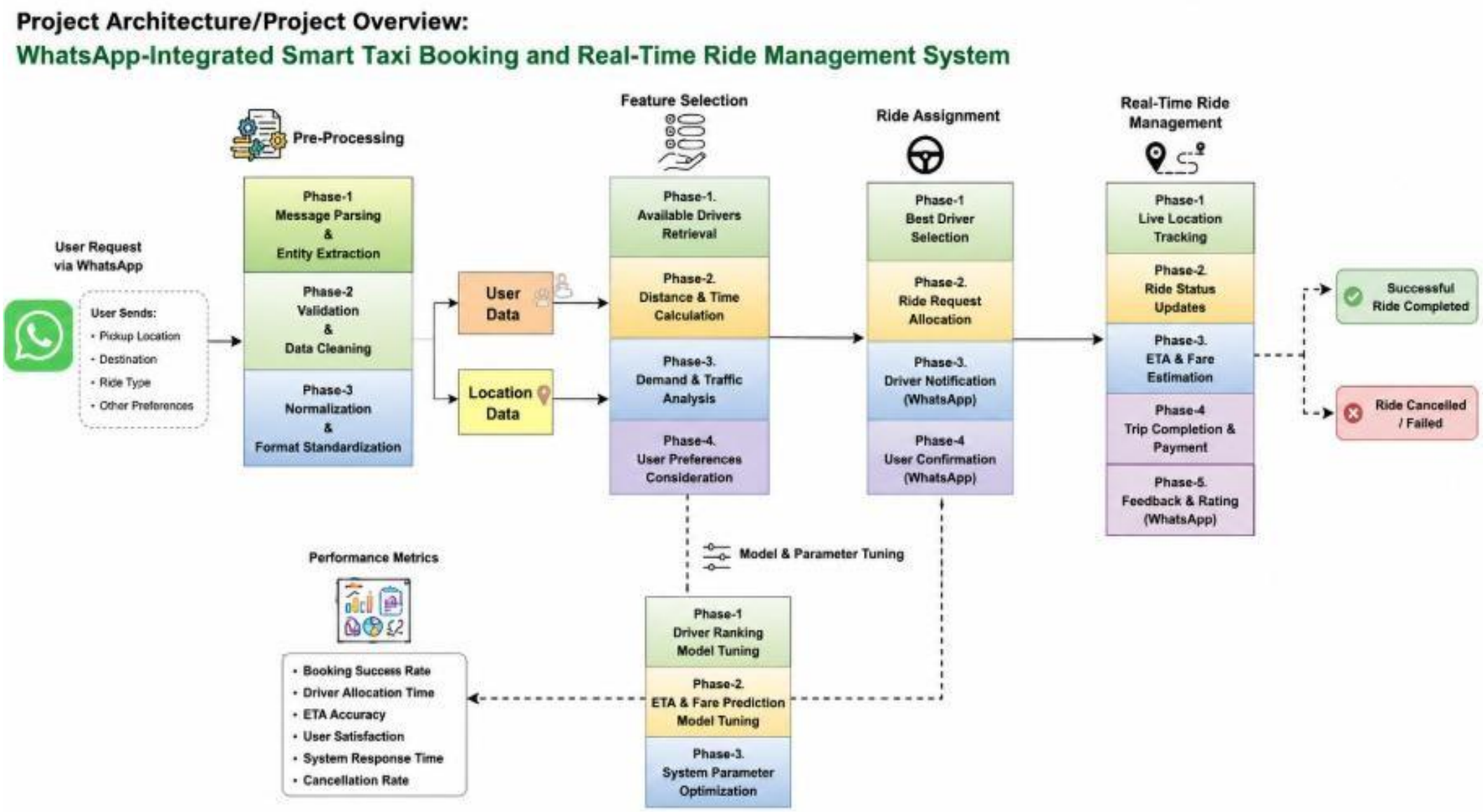
Problem Gap Identification

- Existing taxi systems mainly depend on **mobile apps or websites**.
- Limited integration of **WhatsApp for direct taxi booking**.
- Real-time communication between **passengers and drivers** can be improved.
- Lack of a **simple, app-free booking experience**.
- The proposed system addresses these gaps through **WhatsApp booking, driver assignment, and real-time ride management**.

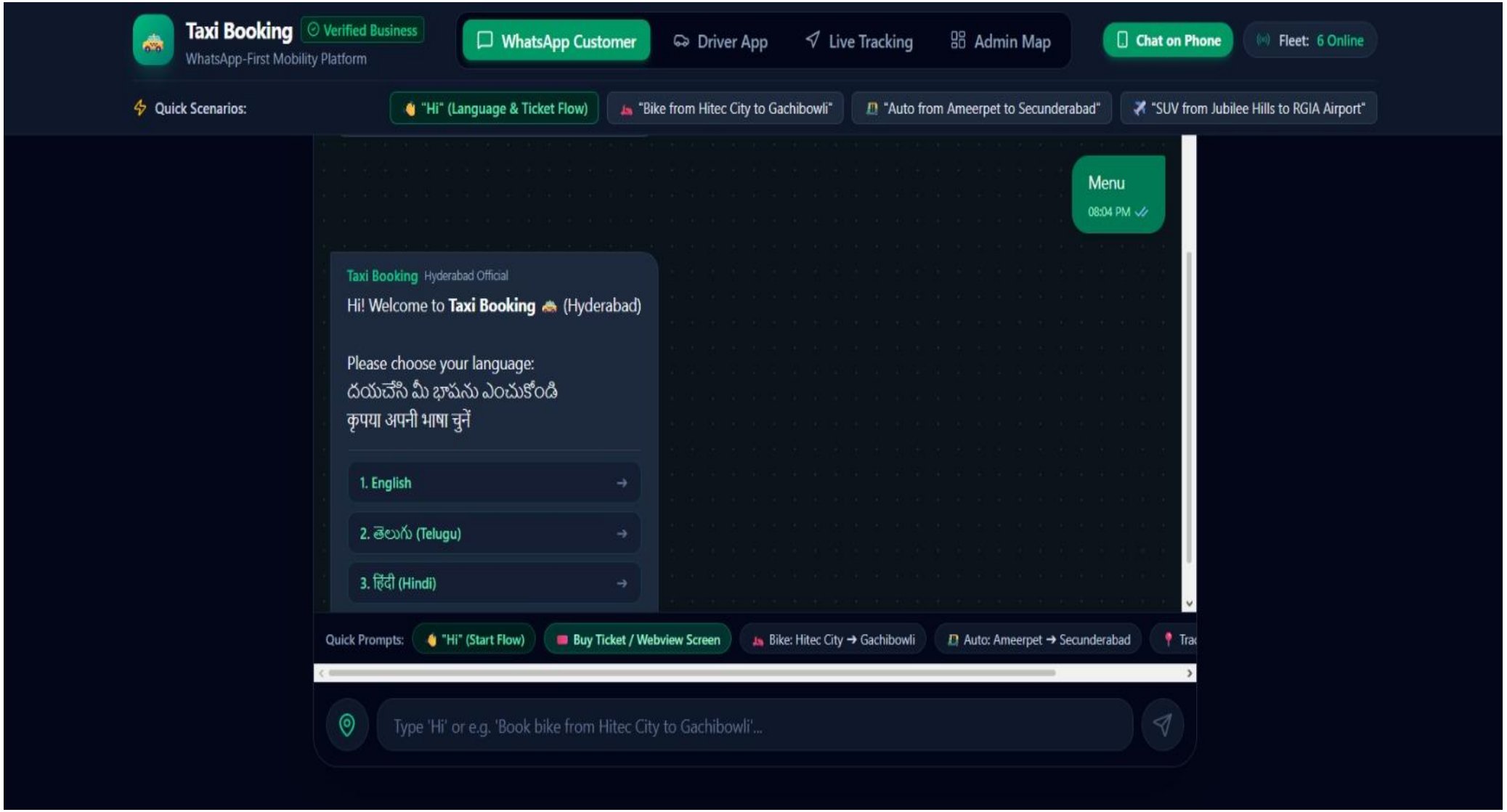
Innovation, Creativity, and Novelty

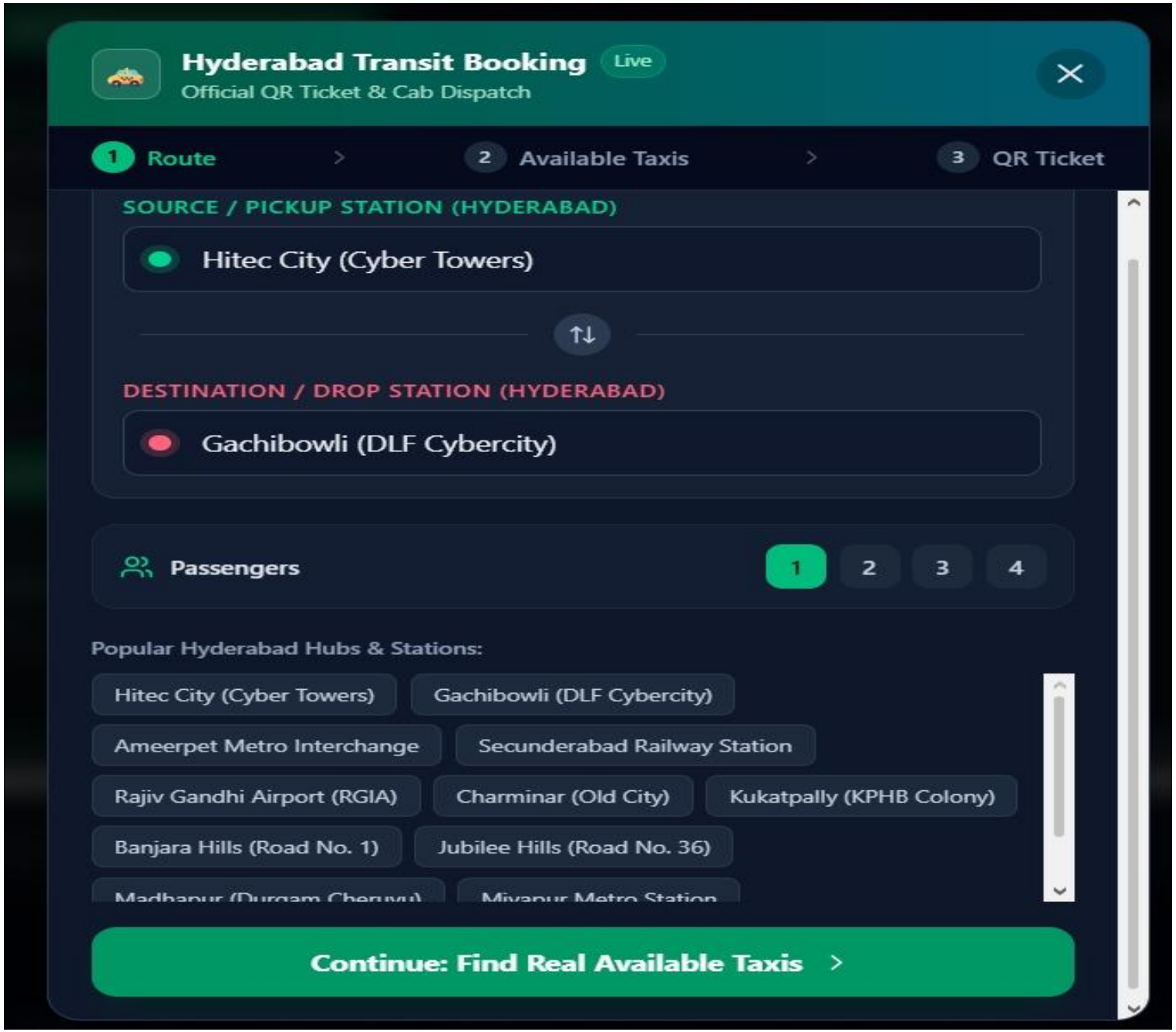
- **WhatsApp-based taxi booking** without requiring a separate app.
- **Real-time ride tracking** and status updates.
- **Automated driver assignment** based on availability and location.
- **Instant WhatsApp notifications** for booking and ride updates.
- Provides a **simple, convenient, and user-friendly** taxi booking experience.

Feasibility and Project Plan



Frontend Implementation




Hyderabad Transit Booking
Live
Official QR Ticket & Cab Dispatch
1 Route
2 Available Taxis
3 QR Ticket
SOURCE / PICKUP STATION (HYDERABAD)
Hitec City (Cyber Towers)
DESTINATION / DROP STATION (HYDERABAD)
Gachibowli (DLF Cybercity)
Passengers
1
2
3
4
Popular Hyderabad Hubs & Stations:
Hitec City (Cyber Towers)
Gachibowli (DLF Cybercity)
Ameerpet Metro Interchange
Secunderabad Railway Station
Rajiv Gandhi Airport (RGIA)
Charminar (Old City)
Kukatpally (KPHB Colony)
Banjara Hills (Road No. 1)
Jubilee Hills (Road No. 36)
Continue: Find Real Available Taxis

Dataset Requirements & Data Collection Plan

Use public data for algorithm prototyping and collect consent-based local data for real deployment.

1. Public benchmark dataset — NYC TLC

NYC Taxi & Limousine Commission trip records contain pickup/drop-off time, taxi-zone locations, trip distance, fare components and payment details. Use it to prototype trip analysis and fare/demand patterns.

Limitation: NYC taxi zones and prices do not represent Hyderabad conditions.

2. Project dataset — Hyderabad pilot

Create a small, consent-based dataset from test rides or simulated bookings in the target service area. Store pickup/drop coordinates, timestamp, trip distance, estimated/actual fare, driver availability and ride status.

Remove names and phone numbers from analytics exports.

Recommended fields for each booking record

booking_id	Unique trip identifier
drop_lat, drop_lng	Destination coordinates
distance_km	Route distance
driver_id, status	Assigned driver and ride status
pickup_lat, pickup_lng	Pickup coordinates
request_time	Booking request timestamp
fare_estimate, fare_final	Estimated and final fare

[Dataset source: NYC TLC Trip Record Data (monthly CSV/Parquet files and data dictionaries)]

Proposed API Integrations

These are integration options; API credentials, billing and platform approval are required before live use.

WhatsApp Business Platform — Cloud API

Purpose: receive booking messages, send confirmations and driver details, and deliver ride-status notifications.
Integration: webhook endpoint receives inbound events; backend sends replies using the Messages API.

Google Maps Platform — Routes API

Purpose: compute route distance, duration and route details for pickup-to-destination journeys.
Optional: Geocoding converts typed addresses to coordinates; Places helps users choose locations.

Backend service — FastAPI / Flask

Expose endpoints such as /webhook/whatsapp, /bookings, /drivers/nearby, /rides/{id} and /fare-estimate. Validate requests and keep API tokens on the server.

Database — PostgreSQL / PostGIS

Persist users, drivers, vehicles, bookings and trip status. PostGIS can support radius-based nearby-driver queries using geographic coordinates.

Official documentation links (clickable in the PowerPoint)

Meta WhatsApp Cloud API — Get Started

Meta WhatsApp Cloud API — Webhooks

Google Maps Routes API

Google Maps Platform Documentation

End-to-End System Workflow

Proposed request-to-completion flow for WhatsApp booking and ride management.

1 Passenger

Sends pickup, destination and passenger count on WhatsApp.

2 WhatsApp Cloud API

Delivers the incoming message to the registered webhook.

3 Backend

Validates input, creates booking and requests route/fare estimate.

4 Driver matching

Finds available drivers near pickup and sends a ride request.

5 Driver response

Accepts or rejects; backend assigns the first eligible driver.

6 Ride updates

Passenger receives driver details and status updates until completion.

Important: use webhook verification, duplicate-message protection and booking status checks so retries do not create duplicate rides.

Database Design — Core Entities

Suggested relational schema; primary keys (PK) and foreign keys (FK) keep bookings consistent.

USERS

user_id (PK)
name, phone_hash
created_at

DRIVERS

driver_id (PK)
name, phone_hash
availability, rating

VEHICLES

vehicle_id (PK)
driver_id (FK)
vehicle_type, plate_no

BOOKINGS

booking_id (PK)
user_id, driver_id (FK)
pickup, drop, status

TRIPS

trip_id (PK)
booking_id (FK)
start/end time, distance

PAYMENTS

payment_id (PK)
booking_id (FK)
estimated/final fare

NOTIFICATIONS

notification_id (PK)
booking_id (FK)
channel, message status

Relationships: one user can create many bookings; one driver can serve many bookings over time; each booking may have a trip, payment and notification records.

Smart Driver Allocation & Fare Estimation

A transparent baseline algorithm can be implemented before introducing advanced ML.

Driver allocation — nearest eligible driver

1. Filter drivers with availability = ONLINE.
2. Calculate distance from pickup to each available driver using coordinates or a route-matrix API.
3. Exclude drivers beyond the configured service radius.
4. Sort by distance (and optionally rating / idle time).
5. Send request to the top candidate; if declined or timed out, try the next candidate.
6. Lock the booking in a database transaction to avoid double assignment.

Fare estimate — configurable formula

Estimated fare = Base fare + (Rate per km × Route distance) + (Rate per minute × Estimated duration) + Applicable fees

Apply minimum fare and clearly disclose any peak-time multiplier before confirmation.

Store the fare estimate at booking time and the final fare after trip completion. Keep rate values configurable in the admin panel.

Evaluation metrics: driver assignment time • average pickup distance • booking success rate • fare-estimation error • cancellation rate

Security, Reliability & Testing Plan

Controls and test cases to include before a real-world pilot.

Security & privacy controls

- Verify Meta webhook challenge and request authenticity.
- Store access tokens and secrets in environment variables.
- Hash or restrict access to phone numbers; do not expose customer data in logs.
- Obtain consent before collecting or sharing live location.
- Apply role-based access for admin and driver accounts.

Reliability controls

- Prevent duplicate bookings using message/event IDs.
- Use booking states: REQUESTED → ASSIGNED → ACCEPTED → IN_PROGRESS → COMPLETED / CANCELLED.
- Add timeout and reassignment when a driver does not respond.
- Log API failures and retry safely; never expose API keys in the frontend.

Minimum test cases

Valid booking	Booking saved and confirmation sent
Missing pickup/destination	Ask passenger for the missing field
No driver available	Inform passenger and offer retry/cancel
Driver rejects request	Try the next eligible driver
Duplicate WhatsApp webhook	Do not create a second booking
Maps API unavailable	Return a clear error or configured fallback

Review-3 Progress & Next Implementation Milestones

Separate completed evidence from planned development; update status after each working demonstration.

Demonstrated in the current presentation

- Frontend booking interface screenshot.
- Admin/dashboard screenshot.
- Project architecture / overview diagram.

To implement / validate next

- Connect WhatsApp Cloud API and verify webhooks.
- Integrate route and fare calculation.
- Implement database persistence and driver assignment.

Phase 1	Database schema, backend booking endpoints and validation
Phase 2	WhatsApp webhook, inbound messages and template/status replies
Phase 3	Maps route calculation, fare estimation and driver matching
Phase 4	Admin/driver workflows, end-to-end testing and pilot evaluation

Expected outcome: a tested prototype with traceable bookings, driver assignment and WhatsApp ride notifications.

References & Development Resources

Official documentation and dataset sources used to prepare the proposed technical plan.

[1] NYC Taxi & Limousine Commission — Trip Record Data	Public trip records, data dictionaries and taxi-zone lookup files.
[2] Meta — WhatsApp Cloud API Get Started	Business Platform setup, phone number and messaging API.
[3] Meta — WhatsApp Cloud API Webhooks	Receiving inbound messages and event notifications.
[4] Google Maps Platform — Routes API	Route distance, duration and route computation.
[5] Google Maps Platform — Documentation	Geocoding, Places and other mapping services.
[6] PostgreSQL — Documentation	Relational database design, constraints and transactions.
[7] PostGIS — Documentation	Geospatial storage and nearby-location queries.

Note: Dataset availability, API quotas, pricing and platform policies may change; check provider documentation before deployment.